

METRIC_GOVERNANCE_GUIDE

METRIC_GOVERNANCE_GUIDE.md — Helios Semantic Layer Governance

The operating manual for the Helios **semantic layer**: the single source of truth for every metric. This guide is normative. If any other prose disagrees with it on a governance rule, fix the prose. The authoritative *data* is `models/semantic/semantic_layer.yaml` ; this guide governs how that file is authored, validated, versioned, and consumed.

0. TL;DR (the contract in nine lines)

1. **No metric exists outside `semantic_layer.yaml`** . Not in agent prompts, not in Python, not in a dashboard.
 2. **`semantic-mcp.build_query` is the only path to SQL**. The LLM never authors raw SQL. (Grounding rule **G1**.)
 3. **An unknown metric or dimension name is a hard error**, never a fallback to free SQL. (Rule **G5**.)
 4. **Every metric carries the full field schema** — definition, SQL, lineage, guardrails, tests, freshness — no partial entries.
 5. **Rates are `SUM(num)/SUM(den)` after grouping**, never an average of per-segment ratios. (Simpson's-paradox defense.)
 6. **All aggregates reconcile to the warehouse within $\leq 0.5\%$** . (Rule **G4**.) Drift beyond that fails the metric and the finding.
 7. **Money uses `*_in_usd` columns only**. Never aggregate a non-USD twin.
 8. **SemVer governs change**. MAJOR breaks consumers; CI eval gate is the regression firewall.
 9. **Guardrails feed the Critic**. Every metric declares how it can be misread; the Critic uses those declarations to refute findings.
-

1. The single-source-of-truth principle

Helios is an autonomous growth-diagnosis engine whose findings are causal-sounding claims about *why* an e-commerce funnel moved. Those claims are only trustworthy if every number traces to one governed definition. The semantic layer is that anchor.

1.1 The three invariants

#	Invariant	Why it holds	What enforces it
I1	No metric exists outside the registry. A metric the registry does not define cannot be queried, charted, or named in a brief.	A second definition site means two truths; two truths means silent drift.	CI referential-integrity compile; per-agent tool allow-lists; audit log proving every <code>sql_text</code> came from <code>semantic-mcp</code> .
I2	<code>build_query</code> is the only SQL path. No agent and no Python module emits raw SQL to the warehouse.	SQL generation (<code>semantic-mcp</code>) is deliberately separated from SQL execution (<code>warehouse-mcp</code>) so a query is governed, cost-checked, and reconciled <i>before</i> a byte is scanned.	<code>warehouse-mcp</code> accepts only SQL whose provenance is a <code>build_query</code> envelope; <code>dry_run</code> precedes every <code>run_query</code> .
I3	0 hallucinated columns/metrics. Emitted SQL can reference only real GA4 columns.	<code>build_query</code> interpolates only registry-defined <code>sql_definition</code> templates and dimension expressions; a hallucinated column (<code>event_params.foo</code>) is not in any template, so it cannot appear in output.	Anti-hallucination is structural, not behavioral — the model literally has no surface to inject a column through.

1.2 The chokepoint chain

```
agent picks canonical names
→ semantic-mcp.build_query (composes governed SQL from registry templates ONLY) ← I1, I3
→ warehouse-mcp.dry_run (byte-budget + schema validation; ≤ 5 GiB/run) ← G3
→ warehouse-mcp.run_query (sole BigQuery client)
→ warehouse-mcp.reconcile (aggregate vs canonical totals; ≤ 0.5% drift) ← G4
→ stats-mcp / experiment-mcp (all math; never in prose) ← G2
→ Critic (adversarial refutation using guardrails)
```

If the chain is bypassed anywhere, the finding is void. The success target is absolute: **0 hallucinated columns/metrics, 100% governed SQL**, verified by the audit log.

2. The per-metric field-schema contract

Every metric mapping — **named and supporting alike** — carries exactly the fields below. There are no optional fields: a `null` is an explicit, reviewed decision, never an omission. The file has a single top-level key `metrics:` whose value is the list of mappings; 2-space indentation; all `sql_definition` / `expr` strings quoted.

2.1 Field-by-field reference

Field	Type	Required	Meaning & rules
<code>metric_name</code>	string (snake_case)	always	The canonical identifier. The ONLY token agents pass. Must be globally unique across all clusters. This is what an "unknown name" check resolves against.
<code>label</code>	string	always	Human-readable display name for the brief and UI (e.g. "Revenue per Session (RPS)"). Cosmetic — never

Field	Type	Required	Meaning & rules
			used for resolution.
category	enum	always	revenue traffic funnel conversion retention acquisition product supporting . Drives grouping/governance. supporting marks a building-block measure that must live in the same cluster that consumes it.
type	enum	always	count sum ratio derived . Determines which of {sql_definition+aggregation_method} / {numerator+denominator} / {expr} is populated.
business_definition	string	always	Plain-English meaning a PM or founder understands without SQL. The contract for <i>what the number means</i> , not how it's computed.
sql_definition	string (quoted)	count/sum (and the canonical form for ratio/derived)	The exact SQL expression. For count / sum it is the additive expression over the grain. For ratio / derived it documents the resolved formula (e.g. "SAFE_DIVIDE(COUNTIF(reached_purchase), COUNT(DISTINCT session_key))"). Must reference only real columns of the metric's grain.
aggregation_method	enum	always	count_distinct countif sum ratio derived window . How the measure aggregates. window is reserved for over-partition expressions like traffic_share .
grain	logical grain	always	One of the registry grains: keys (fct_funnel , fct_sessions , fct_orders , fct_order_items , fct_cohorts , fct_daily_funnel). build_query resolves FROM grains[metric.grain] . A ratio's numerator and denominator must share a grain (or be reconcilable).
entity	enum	always	session user order order_item cohort . The thing being counted/measured; pins the dedup key and guards against grain confusion.
numerator	metric_name null	ratio → set; else null	The defined metric_name forming the numerator. Cross-cluster references are allowed and resolve at the final merge.
denominator	metric_name null	ratio → set; else null	The defined metric_name forming the denominator. Rates are computed SUM(num)/SUM(den) after grouping — never an average of ratios.
expr	string null	derived → set; else null	The derived formula with {metric_name} tokens, e.g. "SAFE_DIVIDE({revenue},{sessions})" . Every {token} must resolve to a defined metric.

Field	Type	Required	Meaning & rules
<code>dimensions_supported</code>	[canonical dim]	always	The whitelist of dimensions this metric may be sliced by. Only canonical dimension names. Slicing by anything outside this list is a hard error. Item-level metrics restrict to item dims; required-dimension metrics (e.g. <code>channel_revenue</code>) must include their required dim.
<code>owner</code>	string	always	Accountable team/role (default <code>analytics-eng</code>). The escalation point for changes and breakages.
<code>version</code>	SemVer	always	<code>MAJOR.MINOR.PATCH</code> of <i>this metric</i> . Bump rules in §4.
<code>caveats</code>	[string]	always	Honest limitations a reader must know (e.g. <code>cac_proxy</code> is NOT real CAC; session-attribution is last-touch).
<code>common_mistakes</code>	[string]	always	The wrong ways analysts actually use this metric (e.g. averaging per-segment rates; mixing grains).
<code>guardrails</code>	map	always	The analytical-safety block consumed by the Critic. Sub-fields: <code>do_not_use_when</code> , <code>misinterpretations</code> , <code>simpsons_paradox_risk</code> , <code>attribution_pitfalls</code> , <code>segmentation_pitfalls</code> . See §7.
<code>root_cause</code>	map	always	RCA lineage. Sub-fields: <code>upstream_drivers</code> (metric_names), <code>downstream_impacts</code> (metric_names), <code>related_dimensions</code> (canonical dims), <code>decomposition_path</code> (prose recipe). See §8.
<code>agents</code>	[agent]	always	Which of Monitor, Decompose, Diagnose, Prescribe, Narrator, Critic consume this metric. See §5.
<code>validation_checks</code>	[string]	always	Runtime assertions (e.g. <code>"0 ≤ value ≤ 1"</code> , <code>"reconciles to warehouse-mcp.reconcile within 0.5%"</code>).
<code>testing</code>	map	always	The test contract. Sub-fields: <code>dbt_tests</code> , <code>quality_checks</code> , <code>anomaly_checks</code> , <code>reconciliation_checks</code> . See §6.
<code>freshness_requirements</code>	string	always	The data SLA (e.g. <code>"daily; available ≤36h after event_date"</code>). See §9.

2.2 The type → population matrix (enforced at compile)

type	populate	leave null	aggregation_method
count	<code>sql_definition</code> , <code>aggregation_method</code>	<code>numerator</code> , <code>denominator</code> , <code>expr</code>	<code>count_distinct</code> or <code>countif</code>
sum	<code>sql_definition</code> , <code>aggregation_method</code>	<code>numerator</code> , <code>denominator</code> , <code>expr</code>	<code>sum</code>

type	populate	leave null	aggregation_method
ratio	numerator , denominator (and canonical sql_definition for docs)	expr	ratio
derived	expr (and canonical sql_definition for docs)	numerator , denominator	derived or window

Any row that violates this matrix fails the referential-integrity compile.

3. The build_query resolver field-name mapping

The semantic layer is authored with **descriptive YAML field names** (for human readability and review) but the semantic-mcp.build_query resolver reads them under **short internal keys**. This is a deliberate authoring-vs-runtime split. Authors must use the descriptive names; the resolver performs the mapping at load time.

YAML authoring field	Resolver internal key	Notes
metric_name	name	The lookup key for get_metric(name) and the entries of build_query(metric=[...]) .
sql_definition	sql	The additive expression interpolated into the SELECT for count / sum metrics.
aggregation_method	agg	Drives wrapping: count_distinct → COUNT(DISTINCT ...), countif → COUNTIF(...), sum → SUM(...), window →over-partition.
dimensions_supported	dimensions	The slice whitelist; build_query(dims=[...]) is validated against it.
numerator / denominator / expr	(used directly)	No rename. numerator / denominator resolve to other metrics' sql /agg and are combined as SAFE_DIVIDE(SUM(num), SUM(den)); expr {tokens} are substituted by referenced metrics' resolved expressions.
grain	grain	Resolved against grains: to the physical relation in the FROM clause.

Authoring rule: always write the descriptive names (metric_name , sql_definition , aggregation_method , dimensions_supported). Never hand-write the short keys — they are an implementation detail of the resolver. The mapping is one-directional and total; if the resolver encounters a short key in the source file it is a compile error (someone authored against the wrong contract).

Composition semantics (what the resolver does with the mapped fields):

- count / sum : emit agg(sql) AS metric_name in the grouped SELECT over grains[grain] .
- ratio : resolve numerator and denominator to their sql / agg , emit SAFE_DIVIDE(SUM(num_expr), SUM(den_expr)) AS metric_name — the **SUM-of-num / SUM-of-den after grouping** form (Simpson's-paradox defense), never AVG(per_row_ratio) .
- derived : substitute each {token} in expr with the referenced metric's resolved measure, emit the resulting expression.

- `window` : emit the `sql_definition` containing the `OVER (...)` clause verbatim (no extra aggregation wrap), e.g. `traffic_share` .
- Every emitted column references only columns present in `grains[grain]` ; any other column is impossible because no template references it (invariant **I3**).

4. Ownership, SemVer, change management, deprecation & aliases

4.1 Ownership

Every metric has an `owner` (default `analytics-eng`). The owner is accountable for the definition's correctness, its tests, and any change. Cross-cluster references (a ratio whose numerator lives in another cluster) create an implicit dependency: the owner of the *referenced* metric must sign off on any change that alters its semantics, because downstream metrics inherit the change.

4.2 SemVer on each metric (`version: MAJOR.MINOR.PATCH`)

Bump	When	Examples	Consumer impact
MAJOR	The number's meaning changes; any consumer could now be wrong.	Change <code>aggregation_method</code> ; change <code>grain / entity</code> ; swap <code>numerator / denominator</code> ; redefine <code>sql_definition</code> so historical values shift; remove a supported dimension.	Breaking. Requires migration plan, eval re-baseline, and a deprecation window if a name is retired.
MINOR	Additive, backward-compatible.	Add a dimension to <code>dimensions_supported</code> ; add an alias; widen <code>caveats / guardrails</code> ; add a <code>validation_check</code> .	Non-breaking; consumers keep working.
PATCH	No behavioral change.	Fix a typo in <code>label / business_definition</code> ; clarify a caveat; tighten a test without changing pass/fail on real data.	None.

`metadata.registry_version` (the file-level SemVer) bumps to the **highest** bump of any metric changed in a release: any MAJOR metric change ⇒ MAJOR registry release.

4.3 Change-management workflow

1. **Propose** — open a PR editing `semantic_layer.yaml` only. No new metric may be introduced anywhere else (invariant **I1**).
2. **Compile** — CI runs the referential-integrity compile (§6.1). Dangling `numerator / denominator / expr / dimension` references fail the build.
3. **Reconcile** — CI recomputes affected metrics against the warehouse; > 0.5% drift fails.
4. **Eval gate** — the smoke eval (12 scenarios) runs on every push; the full benchmark (50 scenarios) on PRs to `main` . A change may **not** drop top-1 root-cause accuracy by > 2 pts or introduce any hallucination.
5. **Review** — owner of the changed metric and owners of any metric that references it approve.
6. **Version** — bump per §4.2; update `CHANGELOG / release notes` ; if a name is being retired, schedule the deprecation window (§4.4).

4.4 Deprecation & aliases

- **Aliases** let a metric be referenced by a legacy or convenience name without forking the definition. The canonical example: `orders` carries **alias** `transactions` — they are the *same* governed measure (`count_distinct` of `order_key` on `fct_orders`). Aliases resolve to the canonical `metric_name` before any query is built; they never create a second definition. An alias addition is a **MINOR** bump.
- **Deprecation** retires a name without instantly breaking consumers. A deprecated metric is marked (e.g. `category: supporting` downgrade or a `caveats` flag `"DEPRECATED: use X"`), kept queryable for a published window (default ≥ 1 full release cycle), and emits a resolver warning. Removal after the window is a **MAJOR** bump.
- **Renames** are an alias + deprecation: add the new `metric_name` , alias the old one to it, deprecate the old name, remove it next MAJOR. Never silently rename — that breaks the audit trail and the eval labels.

5. How each of the six agents consumes the layer

The semantic layer is the *only* metric surface for all six metric-consuming agents (the Orchestrator plans but does not consume metrics). Every agent passes **canonical `metric_name` s** to `semantic-mcp` ; **an unknown metric or dimension name is a hard error (rule G5)** — the server rejects it, the branch hard-stops, and the agent re-plans against `list_dimensions()` / the registry rather than falling back to free SQL. There is no "best-effort" path.

Agent	Model	Consumes (via <code>metric_name</code>)	What it does with the layer	MCP tools
Monitor	Sonnet	Time-series volume/rate/revenue metrics with <code>Monitor</code> in <code>agents[]</code> (<code>sessions</code> , <code>revenue</code> , <code>session_conversion_rate</code> , funnel step counts).	Builds daily series via <code>build_query</code> , feeds <code>detect_anomaly</code> to flag deviations (weekly-seasonality aware). Uses <code>freshness_requirements</code> to know whether today's cell is final.	<code>build_query</code> , <code>dry_run</code> , <code>run_query</code> , <code>detect_anomaly</code>
Decompose	Sonnet	Aggregate rate/efficiency metrics (<code>session_conversion_rate</code> , step rates, <code>engagement_rate</code> , <code>channel_conversion_rate</code>) plus their <code>traffic_share</code> weights.	Splits ΔR into mix vs rate vs interaction via <code>decompose_change</code> , using the metric's <code>root_cause.related_dimensions</code> to choose the split axes. Reads <code>numerator / denominator</code> to know the rate's parts.	<code>build_query</code> , <code>dry_run</code> , <code>run_query</code> , <code>decompose_change</code>
Diagnose	Opus	Nearly all metrics — walks the hypothesis tree.	Drills the largest rate effect down through <code>root_cause.upstream_drivers</code> and <code>related_dimensions</code> ; reconciles each step (<code>reconcile</code>); runs <code>significance_test</code> so every branch carries a p-value.	<code>build_query</code> , <code>dry_run</code> , <code>run_query</code> , <code>reconcile</code> , <code>significance_test</code>

Agent	Model	Consumes (via <code>metric_name</code>)	What it does with the layer	MCP tools
Prescribe	Sonnet	Metrics that become experiment primary/guardrail — <code>conversion rates</code> , <code>aov</code> , <code>revenue</code> , <code>revenue_per_session</code> .	Uses the chosen rate as the experiment primary and revenue metrics as guardrails; feeds <code>power_analysis / runtime_estimate</code> . Reads <code>guardrails.do_not_use_when</code> to avoid an invalid primary.	<code>build_query</code> , <code>dry_run</code> , <code>run_query</code> , <code>power_analysis</code> , <code>runtime_estimate</code> , <code>design_experiment</code>
Narrator	Sonnet	Headline revenue/conversion metrics (<code>revenue</code> , <code>session_conversion_rate</code> , <code>aov</code> , <code>revenue_per_user</code>).	Renders the Decision Brief using each metric's <code>label</code> and <code>business_definition</code> verbatim; never recomputes numbers (all values are tool outputs, rule G2). Surfaces <code>caveats</code> honestly.	<code>build_query</code> (read), <code>render_brief</code> , <code>export</code>
Critic	Opus	Everything it can refute.	Attacks each finding using the metric's <code>guardrails</code> block (mix-vs-rate, Simpson's, attribution, segmentation) and <code>caveats</code> ; cross-checks against the seasonality/launch calendar; downgrades or drops findings that fail.	<code>build_query</code> , <code>dry_run</code> , <code>run_query</code> , <code>reconcile</code> , <code>significance_test</code>

Per-agent allow-lists are enforced (e.g. the Narrator cannot call `run_query`). An agent attempting a metric not in its conceptual scope still resolves through the same registry; the registry's `agents[]` list documents *intended* consumers and is used by tests to ensure each metric is reachable by the agents that need it.

6. Validation & CI pipeline

The CI pipeline is the regression firewall. A change to `semantic_layer.yaml`, `models/`, `agents/`, or `eval/` must clear every gate below before merge.

6.1 Referential-integrity compile (structural)

Run at registry load and in CI. Hard-fails (no warnings-as-pass) on:

- Any `numerator / denominator` that is not a defined `metric_name`.
- Any `{token}` in an `expr` that does not resolve to a defined `metric_name`.
- Any name in `dimensions_supported` (or a required dimension) that is not a canonical dimension.
- A `supporting` measure referenced from a different cluster than the one it is defined in.
- A `type` that violates the population matrix (§2.2).
- A `grain` not present in `grains:`.
- A duplicate `metric_name` or an alias colliding with a real `metric_name`.

This is the structural enforcement of **0 hallucinated columns** (rule G5): a dangling reference cannot ship.

6.2 Reconciliation (numerical, $\leq 0.5\%$)

Each `sum / count` metric and each `ratio`'s numerator/denominator is recomputed and compared to `warehouse-mcp.reconcile` canonical totals (and to the additive `fct_daily_funnel` recompute). **Drift > 0.5% fails the metric and any finding that used it (rule G4).** Revenue must additionally reconcile to the source to the cent (`revenue_reconciles` dbt test) because dollar-at-risk labels depend on it.

6.3 Anomaly & data-quality checks

Per the `testing.anomaly_checks / quality_checks` contract: STL-residual on daily series with weekly seasonality awareness; NULL-spike / duplicate- `transaction_id` / late-shard detection so the Critic can attribute to **data quality** rather than behavior. dbt tests (`not_null`, `accepted_range: 0..1` for rates, uniqueness on keys) run in `dbt build`.

6.4 The eval gate (behavioral)

The offline labeled benchmark (50 scenarios in `helios_eval`) injects known `(metric, segment, time)` perturbations and grades the pipeline's rediscovered root cause against ground truth.

Gate	Trigger	Pass condition
Smoke	every push	12-scenario subset green; no hallucination
Full	PR to <code>main</code>	top-1 root-cause accuracy ≥ 85% (vs ≤ 45% naive baseline); 0 hallucinated columns/metrics ; 100% of findings carry significance + dollar-at-risk ; no > 2-pt top-1 regression vs the green baseline

A metric change that moves a number also moves the eval labels' dependency surface; the eval re-baseline in §4.3 step 4 is mandatory for MAJOR changes.

7. Analytical-guardrail philosophy (and how it feeds the Critic)

Helios's differentiator is **distinguishing mix-shift from rate-change** and refusing to ship a misleading finding. The `guardrails` block on every metric is the machine-readable encoding of *how this metric can mislead*, and it is the Critic's ammunition.

7.1 The four guardrail concerns

Guardrail sub-field	Concern	Canonical defense
<code>simpsons_paradox_risk</code>	A pooled rate moves only because segment weights moved, not behavior.	Rates are <code>SUM(num)/SUM(den)</code> after grouping; <code>decompose_change</code> splits $\Delta R = \text{mix} + \text{rate} + \text{interaction}$; drill the rate effect, not the mix effect.
<code>attribution_pitfalls</code>	Channel credit is wrong. Event-level <code>traffic_source</code> is user first-touch , not session source.	<code>channel_group</code> is last-touch session-scoped by default; never use event-level <code>traffic_source</code> for session analysis. <code>cac_proxy</code> caveats that it is NOT real CAC (no cost data in GA4).

Guardrail sub-field	Concern	Canonical defense
<code>segmentation_pitfalls</code>	A slice is too thin to be significant, or the wrong axis is chosen.	Significance-test every drilled segment; respect <code>dimensions_supported</code> ; small-n segments are flagged, not asserted.
<code>do_not_use_when / misinterpretations</code>	The metric is being applied outside its valid domain or read as something it isn't.	Explicit prohibitions (e.g. don't use <code>cac_proxy</code> as a budget input; don't compare <code>revenue</code> (session-attributed) to <code>gross_revenue</code> (order-grain) as if identical).

7.2 The mix-vs-rate identity (why guardrails exist)

For an aggregate rate $R = \sum_i (w_i \cdot r_i)$ over segments i , the change $t_0 \rightarrow t_1$ decomposes as:

```

mix_effect   =  $\sum \Delta w_i \cdot r_i(t_0)$     # traffic composition changed (NOT a behavior change)
rate_effect  =  $\sum w_i(t_0) \cdot \Delta r_i$     # in-segment behavior changed (the real signal)
interaction  =  $\sum \Delta w_i \cdot \Delta r_i$       # both moved together
 $\Delta R = \text{mix\_effect} + \text{rate\_effect} + \text{interaction}$ 

```

This is computed deterministically in `stats-mcp.decompose_change` (seeded, unit-tested against golden values), never in prose (rule G2). It is how Simpson's paradox is dissolved.

7.3 How guardrails feed the Critic

The Critic loads each finding's underlying metric `guardrails` and attempts refutation: if the "drop in conversion" is a `mix_effect` per `decompose_change`, the Critic invokes `simpsons_paradox_risk` and **downgrades** the finding from rate-change to mix-shift; if the channel claim relies on first-touch attribution, the Critic invokes `attribution_pitfalls`; if the segment is sub-significant, `segmentation_pitfalls`. Findings that match a known seasonality/launch calendar entry within tolerance are **DROP** ped or **DOWNGRADE** d. A finding survives only if the Critic cannot refute it on any declared guardrail.

8. How RCA uses the `root_cause` block

The `root_cause` map turns each metric into a node in the diagnosis hypothesis tree. Diagnose (and Decompose) traverse it deterministically.

Sub-field	Type	Role in RCA
<code>upstream_drivers</code>	[metric_name]	The metrics that <i>cause</i> this one to move. Diagnose climbs upstream to find the proximate driver (e.g. <code>session_conversion_rate</code> ← <code>purchasing_sessions</code> , <code>sessions</code>).
<code>downstream_impacts</code>	[metric_name]	The metrics this one <i>moves</i> . Used to price the movement in dollars (e.g. a step-rate drop's <code>downstream_impacts</code> include <code>revenue</code> , <code>revenue_per_session</code>) and to confirm the blast radius.

Sub-field	Type	Role in RCA
<code>related_dimensions</code>	[canonical dim]	The axes to split on. Decompose runs <code>decompose_change</code> along these (e.g. <code>channel_group × device_category</code>).
<code>decomposition_path</code>	prose	The recipe: "Split by <code>channel_group × device_category</code> ; run <code>decompose_change</code> (mix vs rate vs interaction); drill the largest rate effect."

RCA loop: Monitor flags an anomaly on a metric → Decompose splits along `related_dimensions` and isolates the dominant **rate** effect → Diagnose climbs `upstream_drivers` to the proximate cause, reconciling and significance-testing each step → dollar impact is read off `downstream_impacts` → Prescribe turns the pinned segment into a powered experiment → Critic attempts refutation via guardrails → Narrator writes the brief. The tree is fully specified in the registry, so the traversal is reproducible and auditable.

9. Naming conventions & freshness SLAs

9.1 Naming conventions

- **Metrics:** `snake_case`, semantic and stable (`session_conversion_rate`, not `conv` or `cr`). The `metric_name` is permanent once shipped — renames go through the alias+deprecation path (§4.4).
- **Aliases:** documented, resolve to the canonical name (`transactions` → `orders`). Never a second definition.
- **Dimensions:** only the canonical set — `device_category`, `operating_system`, `browser`, `country`, `region`, `channel_group`, `source`, `medium`, `campaign`, `landing_page`, `item_category`, `item_name`, `item_brand`, `is_new_user`, `session_number_bucket`, `day`, `week`, `month`, `quarter` . No synonyms; the glossary maps analyst-typed synonyms back to canonical names before they reach `semantic-mcp` .
- **Channel groups:** exactly 10 — `Direct`, `Organic Search`, `Paid Search`, `Display`, `Paid Social`, `Organic Social`, `Email`, `Affiliates`, `Referral`, `Other` . No 11th group, no "Paid Other".
- **Supporting measures:** `category: supporting`, defined in the *same cluster* that consumes them, named for what they measure (`purchasing_users`, `cohort_size`, `paid_sessions`).
- **Funnel flags:** `reached_*` (max-downstream monotonic). The `did_*` names are retired.

9.2 Freshness SLAs

Each metric declares `freshness_requirements` . The standard SLA is **daily; available ≤ 36h after event_date** (the GA4 export's intraday-to-final settling window). Consumers honor it: Monitor treats a cell younger than the SLA as non-final and does not anomaly-flag it; the Narrator stamps the brief with the data-as-of date. A stale metric (older than its SLA) fails the freshness check and is excluded from a run rather than reported as fresh.

10. Checklist for adding or changing a metric

Copy this into the PR description and check every box.

10.1 Adding a metric

- ☐ Editing **only** `semantic_layer.yaml` (no metric created anywhere else — invariant I1).
- ☐ `metric_name` is unique, `snake_case`, semantic, and not colliding with any alias.
- ☐ `category` and `type` set; the §2.2 population matrix satisfied (`count / sum` → `sql_definition + aggregation_method`, others null; `ratio` → `numerator + denominator`, `expr` null; `derived` → `expr`, num/den null).
- ☐ `business_definition` is plain-English and PM-readable.
- ☐ `sql_definition / expr` quoted; references only real columns of the declared `grain`; rates use `SUM(num)/SUM(den)`; money uses `*_in_usd`.
- ☐ `grain` exists in `grains:`; `entity` matches the grain; ratio num/den share a reconcilable grain.
- ☐ All `numerator / denominator / expr` tokens resolve to defined metrics; supporting measures live in the consuming cluster.
- ☐ `dimensions_supported` uses only canonical dimensions; required dimensions present (e.g. `channel_group` for channel metrics).
- ☐ `caveats + common_mistakes` written honestly (e.g. proxy metrics flagged as NOT the real thing).
- ☐ `guardrails` complete: `do_not_use_when`, `misinterpretations`, `simpsons_paradox_risk`, `attribution_pitfalls`, `segmentation_pitfalls`.
- ☐ `root_cause` complete: `upstream_drivers`, `downstream_impacts`, `related_dimensions`, `decomposition_path`.
- ☐ `agents[]` lists the intended consumers (Monitor/Decompose/Diagnose/Prescribe/Narrator/Critic).
- ☐ `validation_checks + testing` (`dbt_tests`, `quality_checks`, `anomaly_checks`, `reconciliation_checks`) defined; rates carry `0..1` bounds.
- ☐ `owner` and `version` (`1.0.0` for new) set; `freshness_requirements` declared.
- ☐ CI green: referential-integrity compile, reconciliation $\leq 0.5\%$, anomaly/quality checks, smoke eval (no hallucination, no > 2 -pt regression).

10.2 Changing an existing metric

- ☐ Correct SemVer bump chosen (MAJOR if meaning/number changes; MINOR if additive; PATCH if cosmetic — §4.2).
- ☐ Downstream metrics that reference this one identified; their owners pulled into review.
- ☐ If retiring/renaming a name: alias added, deprecation window published, removal scheduled for next MAJOR (§4.4).
- ☐ Reconciliation re-run; revenue still matches source to the cent if revenue-touching.
- ☐ **Full eval re-baselined for MAJOR changes**; top-1 accuracy $\geq 85\%$, 0 hallucinations, no > 2 -pt regression.
- ☐ `metadata.registry_version` bumped to the highest per-metric bump in the release; CHANGELOG/release notes updated.
- ☐ CLAUDE.md / Bible Reference Card updated if a canonical name changed (docs in lockstep).

This guide governs the file. The file (`models/semantic/semantic_layer.yaml`) governs every metric. Every Helios agent governs nothing but consumes the file. That is the whole chain — keep it unbroken.